

Квантизация на LLM

Как голям модел може да се изпълнява на стандартна видеокарта

Проблемът - размер на модела

Съвременните Големи езикови модели съдържат милиарди параметри. Всеки параметър е число, което традиционно се съхранява в 32 битов float (fp32). Това прави дори и модели от среден клас с малко параметри, практически неизпълними на стандартен хардуер.

** Нужда от намаляване на паметта без значителна загуба на точност*

~280 GB

Llama 3 70B | при fp32

~124 GB

Gemma 4 31B | при fp32

24 GB

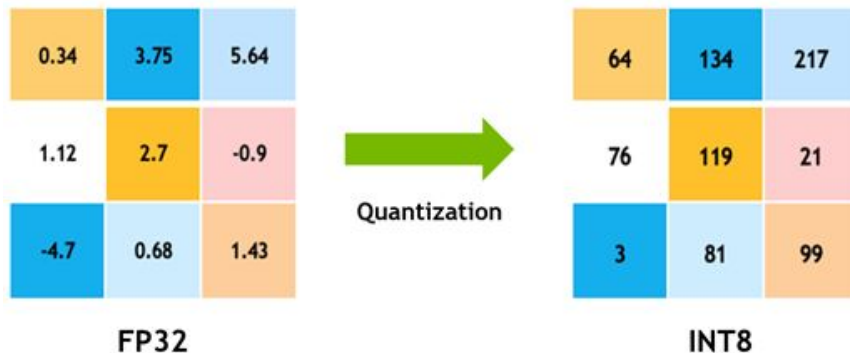
Nvidia RTX 5090

8 GB

Nvidia RTX 4060

Какво е квантизация?

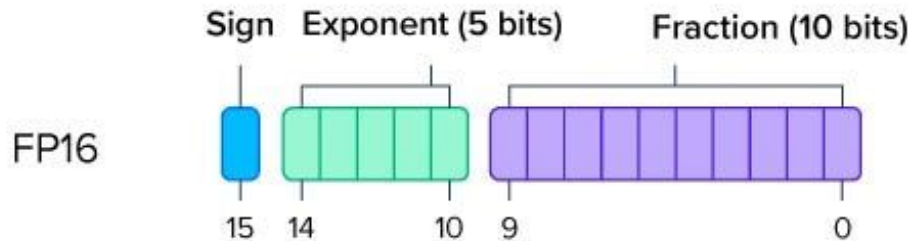
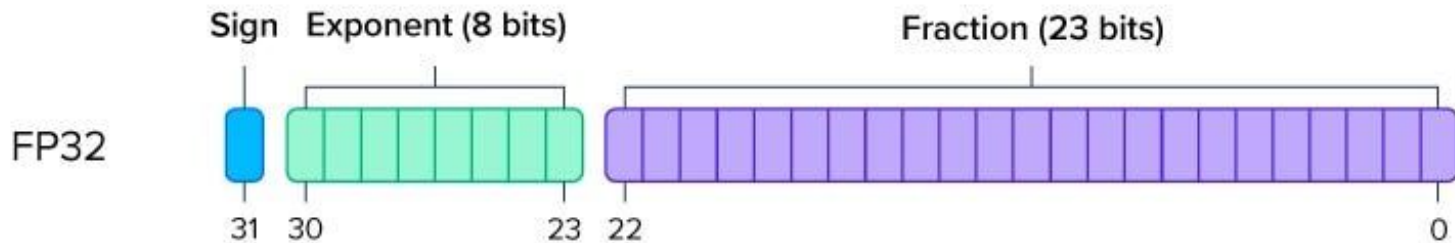
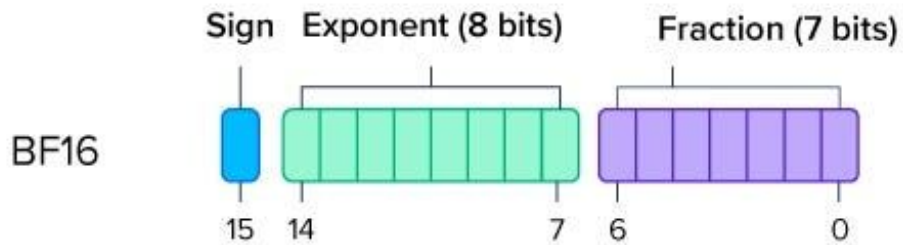
Квантизацията преобразува числата с плаваща запетая (float) в числа с по-малко битове, тези числа представляват теглата на модела. По-малко битове = по-малко памет и по-евтино изпълнение.



Какво е квантизация?

Ключовата идея е, че FP32 може да представи над 4 милиарда различни стойности, но теглата на реален модел са концентрирани в тесен диапазон. INT4 с 16 стойности е достатъчен да улови тази информация с ~93–96% запазено качество. Квантизацията мащабира и закръгля всяко число към най-близката налична стойност.

Как работи квантизацията			
Колко уникални стойности може да представи всеки формат			
FP32	$2^{32} =$	4 294 967 296 стойности	изключително точен
INT8	$2^8 =$	256 стойности	много добър
INT4	$2^4 =$	16 стойности	добър
INT2	$2^2 =$	4 стойности	слаб



Видове квантизация: PTQ vs QAT

Post-Training Quantization (PTQ) — след обучение

- Взема вече обучен модел и конвертира теглата без преобучение
- Калибриране с малък набор данни (~128 примера) за намиране на scale факторите
- Бързо — минути до часове, работи с всеки публичен модел
- Малка загуба на качество (~4–7% при INT4)
- Практиката: почти всичко локално е PTQ

Quantization-Aware Training (QAT) — по време на обучение

- Симулира квантизацията по време на самото обучение (fake-quant операции)
- Моделът се адаптира към по-ниска точност — запазва ~98% качество при INT4
- Изисква достъп до обучителни данни, десетки GPU и дни обучение
- Използва се от Meta и Google за официалните квантизирани версии

PTQ INT4 ≈ 94% качество | QAT INT4 ≈ 98% качество

Популярни методи за квантизация

GPTQ (Gradient Post-Training Quantization)

Layer-by-layer PTQ с Hessian матрица. Стандарт за GPU inference при INT4.

Файлове: model-4bit-128g.safetensors

GGUF / GGML (GPT-Generated Unified Format)

Формат за llama.cpp и Ollama. Поддържа CPU + GPU offload. Файлове:

model.Q4_K_M.gguf

AWQ

Activation-aware Weight Quantization. Защищава важни тегла при outlier активации.

По-малка загуба от GPTQ при INT4.

bitsandbytes

Hugging Face интеграция. Load-time квантизация с 2 реда код. Поддържа INT8 и NF4 с QLoRA fine-tuning.

Кой метод за кой случай:

- Домашен PC → GGUF + Ollama или LM Studio
- Python / research → bitsandbytes или AWQ
- Production inference → GPTQ + vLLM или AWQ

TurboQuant - квантизация на KV кеша

Google Research, март 2026 - нова категория квантизация

Какво е различното

- GPTQ / AWQ / GGUF квантизират теглата на модела (weights) - веднъж, преди inference
- TurboQuant квантизира KV кеша - паметта за всички предишни токени по време на inference
- При дълъг контекст KV кешът може да заема повече памет от самия модел

Резултати

- До 6 пъти по-малък KV кеш без загуба на качество (тествано на Llama 3.1, Gemma, Mistral)
- 4 пъти повече контекст в същата GPU памет - директна полза при дълги разговори
- Комбинира се с GGUF / GPTQ - INT4 тегла + TurboQuant KV кеш = максимална ефективност